Software License Compliance & Audit-Defense (Effective License Position)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Software License Compliance & Audit-Defense (Effective License Position) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Software License Compliance & Audit-Defense (Effective License Position) 5

What an Effective License Position is . . . 5

Why this matters (the why lives here) . . . 5

How it works (the reconciliation model) . . . 5

What you can control . . . 6

The four named artifacts (reuse these names verbatim everywhere) . . . 6

Common compliance gaps (what reconciliation surfaces) . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Software License Compliance & Audit-Defense (Effective License Position) 7

2.1 Entitlement system of record . . . 7

2.2 Proofs of purchase / entitlement evidence . . . 7

2.3 Deployment / usage discovery data . . . 7

2.4 Product use rights for the in-scope product . . . 7

2.5 A reconciliation method . . . 8

2.6 Owner sign-off . . . 8

Checklist . . . 8

3. Implementation Guide — Software License Compliance & Audit-Defense (Effective License Position) 8

Step 1 — Choose the pilot scope (one publisher/product) . . . 9

Step 2 — Build the entitlement baseline . . . 9

Step 3 — Collect the deployment inventory . . . 9

Step 4 — Normalize both sides . . . 9

Step 5 — Reconcile to an Effective License Position . . . 9

Step 6 — Register the gaps . . . 10

Step 7 — Close the gaps . . . 10

Step 8 — Re-reconcile and confirm . . . 10

Step 9 — Assemble the audit package . . . 10

Step 10 — Expand to more publishers . . . 10
5. Testing Guide — Software License Compliance & Audit-Defense (Effective License Position)
Test 1 — Both inputs are complete . . . 11
Test 2 — Normalization joins cleanly . . . 11
Test 3 — The ELP reconciles . . . 11
Test 4 — The gap register is complete . . . 11
Test 5 — Remediation previews safely . . . 11
Test 6 — Gaps close on re-reconciliation . . . 11
Test 7 — Audit package is defensible . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Software License Compliance & Audit-Defense (Effective License Position) 12
A. The ELP does not reconcile / numbers look wrong . . . 12
B. Entitlements cannot be proven . . . 12
C. Deployment data is incomplete . . . 13
D. Gaps are not closing . . . 13
E. Remediation removed nothing . . . 13
F. Quick reference . . . 13
G. When to escalate . . . 13
7. Best Practices — Software License Compliance & Audit-Defense (Effective License Position)
8. Deliverables Summary — Software License Compliance & Audit-Defense (Effective License Position) 14
Documentation . . . 14
Named artifacts produced . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 15
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Software License Compliance & Audit-Defense (Effective License Position)

A software audit letter should not trigger a fire drill. Without an Effective License Position (ELP) — your purchased entitlements reconciled against what you actually deploy and use — you cannot prove compliance, and any shortfall becomes an expensive settlement found on the vendor's terms, not yours. This package builds a defensible ELP and an audit-ready posture the repeatable way: reconcile what you own against what you run, close gaps before they are found, and assemble the proof so you walk into any audit with the numbers already in hand.

This is a complete, do-it-today package for a Software Asset Management (SAM) analyst with no prior ELP experience. Every step is "do this" — gather entitlements, collect deployment, normalize, reconcile, close gaps, and assemble the audit package — proven on one publisher/product pilot before you scale.

What you get

- A plain-language Solution Overview of what an ELP is, the entitlement-versus-deployment reconciliation model, common compliance gaps, and the audit-defense posture.
- A Requirements Guide with verify and implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps in the required order.
- Five PowerShell scripts for deployment discovery, reconciliation validation, an ELP and gap report, safe remediation of over-deployment, and a reconciliation-scope group builder.
- Testing and Troubleshooting guides for a confident, repeatable rollout.

What this delivers

A reconciled `Effective License Position – <Publisher/Product> – ELP` with every shortfall and surplus identified in a `Gap Register – <Publisher/Product> – Reconciled`, gaps closed through a documented remediation plan, and an `Audit Package – <Publisher/Product> – Defense` that bundles the entitlement proofs, deployment evidence, reconciliation, and product-use-rights interpretation an auditor would ask for.

The deployment model

Assess to Configure to Pilot to Validate to Expand — proven on a single publisher/product pilot before you expand to more publishers.

Outcome

A defensible Effective License Position — entitlements reconciled to deployment, gaps closed, and an audit package ready — so a vendor audit holds no surprises.

1. Solution Overview — Software License Compliance & Audit-Defense (Effective License Position)

What an Effective License Position is

An Effective License Position (ELP) is a point-in-time statement, per publisher and product, of what you are *entitled* to use (purchased licenses, subscriptions, and the rights attached to them) reconciled against what you have *deployed and used* (installs, assignments, and active usage). The difference is your compliance position: a shortfall (you run more than you own) is exposure; a surplus (you own more than you run) is wasted spend. The ELP is the single number an auditor — and your finance team — actually cares about.

Why this matters (the why lives here)

A vendor software audit is a financial event. When the auditor's measurement is the only measurement in the room, you negotiate from behind: unproven entitlements are treated as zero, every install is assumed licensable, and shortfalls settle at list price plus back-maintenance and penalties. Building your own ELP *first* flips that: you know the gaps before the auditor does, you close the cheap ones quietly, and you arrive with reconciled numbers and the proofs to defend them. The same ELP also surfaces surplus you can reharvest or stop renewing — so the work pays for itself even without an audit.

How it works (the reconciliation model)

1. Gather entitlements into an `Entitlement Baseline – <Publisher/Product> – Source` — every license

and subscription you own, with quantity, edition/version, metric (per-user, per-device, per-core), and a proof of purchase for each line.

1. Collect deployment and usage into a `Deployment Inventory – <Publisher/Product> – Discovered` —

what is installed, assigned, or actively used, from your discovery and identity sources.

1. Normalize both sides so they compare on the same key: same product name, same edition, same

counting metric (a per-core product counts cores, not installs).

1. Reconcile the two into an `Effective License Position – <Publisher/Product> – ELP`: entitled

quantity minus consumed quantity, per product line, applying the product use rights (downgrade, second-use, virtualization, BYOL) that change the count.

1. Record every difference in a `Gap Register – <Publisher/Product> – Reconciled` — shortfalls

(compliance exposure) and surpluses (reharvest/renewal candidates), each with an owner and action.

1. Close the gaps (true-up, reharvest, reassign, or correct the data) and re-reconcile.

2. Assemble an `Audit Package – <Publisher/Product> – Defense` — the entitlement proofs, deployment

evidence, normalization notes, product-use-rights interpretation, the ELP, and the gap closure record — so the position is defensible on demand.

What you can control

- Which publishers/products are in scope and the reconciliation cadence (per product family).
- The counting metric and product use rights applied to each line (the most common source of dispute).
- How shortfalls are closed: true-up purchase, reharvest of unused, reassignment, or a data fix.
- The contents and freshness of the audit package, so you are always audit-ready, not audit-reactive.

The four named artifacts (reuse these names verbatim everywhere)

- `Entitlement Baseline – <Publisher/Product> – Source` — what you own, with proofs.
- `Deployment Inventory – <Publisher/Product> – Discovered` — what you run, with sources.
- `Effective License Position – <Publisher/Product> – ELP` — the reconciliation workbook.
- `Gap Register – <Publisher/Product> – Reconciled` — shortfalls and surpluses with actions.
- `Audit Package – <Publisher/Product> – Defense` — the bundled, defensible evidence set.

Common compliance gaps (what reconciliation surfaces)

- Over-deployment — more installs/assignments than entitlements (direct exposure).
- Edition mismatch — a Standard entitlement running a Professional install.
- Wrong metric — counting installs for a per-core or per-processor product.
- Unproven entitlements — licenses you believe you own but cannot evidence with a purchase record.
- Stale assignments — licenses assigned to disabled or departed users (reharvest candidates).
- Virtualization/second-use rights not applied — over-counting because rights were ignored.

Limitations (be honest with buyers)

- An ELP is only as good as its two inputs: incomplete entitlement records or partial discovery produce a position you cannot defend. Requirements G3 and G4 are non-negotiable.
- Product use rights vary by publisher and contract; this package gives the repeatable method and where to record the interpretation, not legal advice on a specific contract clause.
- Reconciliation is point-in-time; deployment drifts, so the cadence (not a one-off) is what keeps you audit-ready.

- This package covers the SAM reconciliation and audit-defense process; it does not replace a full discovery tool where one is required for accurate deployment data.

The standard deployment model

> *Build the entitlement baseline and deployment inventory, normalize, reconcile to an ELP, register and close gaps, then assemble the audit package — proven on one publisher/product pilot before expanding. Model: Assess to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — Software License Compliance & Audit-Defense (Effective License Position)

Verify every requirement before reconciling. Each entry: Description, Why, Verify, and Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md`.

2.1 Entitlement system of record

Description: An authoritative record of purchased entitlements for the in-scope publisher/product — quantity, edition/version, counting metric, and renewal date — that becomes the `Entitlement Baseline – <Publisher/Product> – Source`. Why: An ELP is meaningless without a known, authoritative entitlement count to reconcile against. Verify: You can locate the current entitlement quantity, edition, and renewal date for the pilot product. Implement-if-missing: Stand up the records repository (SAM tool, CMDB, or a controlled spreadsheet) and import current purchase orders, license certificates, and true-up statements.

2.2 Proofs of purchase / entitlement evidence

Description: The supporting documents that prove each entitlement line — purchase orders, license certificates, true-up statements, contract/agreement numbers. Why: In an audit, an entitlement you cannot evidence is counted as zero; proofs are what make the ELP defensible. Verify: For a sample of entitlement lines, you can produce the matching purchase record. Implement-if-missing: Collect proofs from procurement, the publisher portal, and finance; store one proof reference per entitlement line in the `Entitlement Baseline – <Publisher/Product> – Source`.

2.3 Deployment / usage discovery data

Description: A source of what is actually installed, assigned, or used for the in-scope product — Microsoft Intune, Configuration Manager, a discovery tool, or sign-in/usage telemetry — exported to the `Deployment Inventory – <Publisher/Product> – Discovered`. Why: Reconciling entitlements against real deployment/usage is the core of the ELP. Verify: You can export an installed-software or license-assignment report for the in-scope product; run `LICCOMP-Discovery.ps1`. Implement-if-missing: Enable Intune/Configuration Manager inventory or onboard a discovery tool; for SaaS, enable usage/sign-in reporting and export per-user assignments.

2.4 Product use rights for the in-scope product

Description: The licensing rules that govern how the product is counted — counting metric (per-user, per-device, per-core/processor), downgrade rights, second-use, virtualization, and BYOL. Why: Applying the wrong metric or ignoring a right is the most common source of an inaccurate ELP and of audit disputes. Verify: You can state the counting metric and the applicable rights for the pilot product, with the contract/agreement clause they come from. Implement-if-missing: Obtain the publisher's product terms and your contract; record the metric and rights interpretation as a note in the `Effective License Position – <Publisher/Product> – ELP`.

2.5 A reconciliation method

Description: A repeatable way to normalize and reconcile both sides on the same key — the `Effective License Position – <Publisher/Product> – ELP` workbook (Excel/Power Query) or an equivalent. Why: Entitlements and deployment rarely share product names or metrics; normalization is what makes the subtraction valid. Verify: A workbook exists that joins entitlements to deployment on a normalized product key; run `LICCOMP-Validation.ps1` to confirm the two input sets are present and reconcilable. Implement-if-missing: Build the ELP workbook with a normalization mapping table (raw name to canonical product/edition) and a reconciliation sheet; or use your SAM tool's reconciliation feature.

2.6 Owner sign-off

Description: Named owners across SAM, procurement, and finance who approve the ELP and authorize gap closure (true-up, reharvest, reassignment). Why: Closing a shortfall spends money and closing a surplus changes renewals — both require an auditable decision, not an analyst acting alone. Verify: A gap action can be raised and routed to the correct owner for approval. Implement-if-missing: Define the SAM analyst, approver, and procurement liaison roles and a documented intake/approval step for gap actions.

Checklist

- Entitlement system of record located — `Entitlement Baseline – <Publisher/Product> – Source`
- Proofs of purchase collected for each entitlement line
- Deployment/usage discovery exported — `LICCOMP-Discovery.ps1`
- Product use rights and counting metric recorded for the in-scope product
- Reconciliation workbook/method ready — `LICCOMP-Validation.ps1`
- Owners named for ELP sign-off and gap closure
- Pilot scope chosen (one publisher/product) — `LICCOMP-Set-ScopeTargeting.ps1`

3. Implementation Guide — Software License Compliance & Audit-Defense (Effective License Position)

Follow these steps in order. Do the work for one publisher/product first (the pilot), validate it, then expand to more publishers.

Model: Build both inputs, normalize, reconcile to an ELP, register and close gaps, assemble the audit package — proven on a single publisher/product before you scale.

Step 1 — Choose the pilot scope (one publisher/product)

1. Pick one high-value, well-understood product as the pilot (for example, a single publisher's flagged product with a known entitlement count and a clear discovery source).

1. Define the population to reconcile (the users/devices that could consume it). Run `LICCOMP-Set-ScopeTargeting.ps1 -ProductName "<Product>" -WhatIf` to preview the in-scope group, then run it without `-WhatIf` to build the reconciliation-scope group.

Step 2 — Build the entitlement baseline

1. From the entitlement system of record, export every line for the in-scope product: quantity, edition/version, counting metric, renewal date, and a proof-of-purchase reference per line.

1. Save it as the `Entitlement Baseline – <Publisher/Product> – Source`.

2. Confirm every line has a proof reference (Requirement 2.2); flag any line you cannot evidence as "unproven" — it counts as zero until evidenced.

Step 3 — Collect the deployment inventory

1. Export installs/assignments/usage for the in-scope product from your discovery and identity sources. For installed software, run `LICCOMP-Discovery.ps1 -CsvPath .\deployment.csv` against your inventory export, or export from Intune/Configuration Manager. For SaaS, export per-user license assignments and recent sign-in/usage.

1. Save it as the `Deployment Inventory – <Publisher/Product> – Discovered`.

Step 4 — Normalize both sides

1. In the `Effective License Position – <Publisher/Product> – ELP` workbook, build a normalization mapping table: raw product/edition names to one canonical product, edition, and counting metric.

1. Map both the baseline and the inventory through it so they compare on the same key.

2. Convert the deployment count to the entitlement's metric (for a per-core product, count cores; for per-user, count distinct active users — not installs).

Step 5 — Reconcile to an Effective License Position

1. In the ELP workbook, for each canonical product line compute: entitled quantity minus consumed quantity, after applying the product use rights from Requirement 2.4 (downgrade, second-use, virtualization, BYOL).

1. Record the metric and rights interpretation as a note on each line (with the contract clause).

2. Run `LICCOMP-Validation.ps1` to confirm both inputs are present, normalized, and reconcilable

before you trust the numbers.

Step 6 — Register the gaps

1. Run `LICCOMP-Report.ps1 -ElpPath .\elp.csv -CsvPath .\gap-register.csv` to produce the ELP summary

and the gap list.

1. Record every difference in the `Gap Register – <Publisher/Product> – Reconciled`: shortfalls

(compliance exposure) and surpluses (reharvest/renewal candidates), each with a cause, an owner, and a planned action.

Step 7 — Close the gaps

1. For each shortfall, choose the closure: reharvest unused assignments, reassign, correct the data,

or true-up (purchase). Get owner sign-off (Requirement 2.6) before any spend.

1. For safe reclamation of clear over-deployment (stale/disabled-user assignments), run

`LICCOMP-Remediation.ps1` — it defaults to `-WhatIf` and only previews; re-run with `-WhatIf:$false` after sign-off to act.

1. For surpluses, route to procurement as reharvest or renewal-reduction candidates.

Step 8 — Re-reconcile and confirm

1. Refresh the deployment inventory and re-run Steps 4 to 6.

2. Confirm the `Gap Register – <Publisher/Product> – Reconciled` shortfalls are closed or have an

approved, dated action. Complete the Testing Guide.

Step 9 — Assemble the audit package

1. Bundle into the `Audit Package – <Publisher/Product> – Defense`: the entitlement baseline with

proofs, the deployment inventory with its source/date, the normalization notes, the product use rights interpretation, the ELP, and the gap-closure record.

1. Have the named owners sign off the package so the position is defensible on demand.

Step 10 — Expand to more publishers

1. Once the pilot passes, repeat Steps 1 to 9 for the next publisher/product, reusing the same five

artifact names and the same workbook structure.

1. Set the reconciliation cadence (per product family) so the ELP stays current, not a one-off.

Order of operations

```
Pilot scope -> Entitlement baseline -> Deployment inventory -> Normalize -> Reconcile (ELP) -> Gap register -> Close gaps -> Re-reconcile -> Audit package -> Expand
```

5. Testing Guide — Software License Compliance & Audit-Defense (Effective License Position)

Run after the pilot reconciliation (Implementation Step 8) and after each reconciliation cycle.

Test 1 — Both inputs are complete

Do: Open the `Entitlement Baseline – <Publisher/Product> – Source` and the `Deployment Inventory – <Publisher/Product> – Discovered`; run `LICCOMP-Validation.ps1`. Expected: Both sets are present for the in-scope product, with no empty quantity, edition, or metric fields; every entitlement line has a proof reference (or is flagged "unproven").

Test 2 — Normalization joins cleanly

Do: In the `Effective License Position – <Publisher/Product> – ELP` workbook, check the normalization mapping; run `LICCOMP-Validation.ps1`. Expected: Every raw product/edition name maps to one canonical line; no unmapped rows; deployment is counted in the entitlement's metric.

Test 3 — The ELP reconciles

Do: Review the ELP line math (entitled minus consumed, after product use rights); run `LICCOMP-Report.ps1`. Expected: Each product line shows a clear position; rights and metric notes are present; the totals match the report.

Test 4 — The gap register is complete

Do: Open the `Gap Register – <Publisher/Product> – Reconciled`. Expected: Every shortfall and surplus from the ELP appears, each with a cause, an owner, and a planned action; no gap is unassigned.

Test 5 — Remediation previews safely

Do: Run `LICCOMP-Remediation.ps1` (default `-WhatIf`). Expected: It reports the over-deployment/stale assignments it would reclaim and makes no change; nothing is removed until re-run with `-WhatIf:$false` after sign-off.

Test 6 — Gaps close on re-reconciliation

Do: After closing actions, refresh the inventory and re-run `LICCOMP-Report.ps1`. Expected: Shortfalls are closed or carry an approved, dated action; the position improves.

Test 7 — Audit package is defensible

Do: Open the `Audit Package – <Publisher/Product> – Defense`. Expected: It contains the entitlement proofs, deployment evidence with source/date, normalization notes, product-use-rights interpretation, the ELP, the gap-closure record, and owner sign-off.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 4 and Test 7 pass for the pilot publisher/product; shortfalls have an approved action
Expand pass	Tests 1 to 7 pass for each added publisher/product before it is declared audit-ready

6. Troubleshooting Guide — Software License Compliance & Audit-Defense (Effective License Position)

Use `LICCOMP-Discovery.ps1` and `LICCOMP-Validation.ps1` for the input data state, `LICCOMP-Report.ps1` for the ELP and gaps, and `LICCOMP-Remediation.ps1` (default `-WhatIf`) for safe reclamation of clear over-deployment.

A. The ELP does not reconcile / numbers look wrong

Symptom	Cause	Remediation
Entitled and consumed counts seem incomparable	Different counting metric on each side	Normalize the deployment count to the entitlement metric (per-core, per-device, per-user); re-run `LICCOMP-Report.ps1`
Many unmapped deployment rows	Normalization mapping incomplete	Add raw-name to canonical mappings in the ELP workbook; re-run `LICCOMP-Validation.ps1`
Shortfall larger than expected	Product use rights not applied	Apply downgrade/second-use/virtualization/BYOL rights per Requirement 2.4 and re-reconcile

B. Entitlements cannot be proven

- An entitlement line with no purchase record counts as zero in an audit. Mark it "unproven" in the

`Entitlement Baseline – <Publisher/Product> – Source`, then recover the proof from procurement, the publisher portal, or finance before relying on the line.

C. Deployment data is incomplete

Symptom	Cause	Remediation
Known devices missing from the inventory	Discovery source not covering the population	Confirm the scope group (`LICCOMP-Set-ScopeTargeting.ps1`); widen the discovery export
Installs present but no usage	Usage telemetry not enabled (SaaS)	Enable sign-in/usage reporting and export recent activity before counting active users

D. Gaps are not closing

• A shortfall stays open when no owner is assigned or no closure is chosen. Ensure every gap in the `Gap Register – <Publisher/Product> – Reconciled` has an owner and one of: reharvest, reassign, data fix, or true-up — and that owner sign-off (Requirement 2.6) is recorded before spend.

E. Remediation removed nothing

• `LICCOMP-Remediation.ps1` defaults to `-WhatIf` and only previews. After owner sign-off, re-run with `-WhatIf:$false` to act. Confirm the input list (stale/disabled-user assignments) is correct first — remediation is for clear over-deployment, not disputed lines.

F. Quick reference

```
.\LICCOMP-Discovery.ps1 -CsvPath .\deployment.csv     # collect installed-software inventory
.\LICCOMP-Validation.ps1 -EntitlementCsv .\ent.csv -DeploymentCsv .\dep.csv   # confirm inputs reconcilabl
.\LICCOMP-Report.ps1 -ElpPath .\elp.csv -CsvPath .\gap-register.csv           # ELP summary + gap list
.\LICCOMP-Remediation.ps1 -InputCsv .\stale.csv       # preview reclamation (WhatIf by default)
```

G. When to escalate

If both inputs are complete and proven, the normalization maps cleanly, and product use rights are applied, but the position is still disputed (for example, a publisher contests your metric or rights interpretation), capture the ELP, the proofs, and the contract clause in the `Audit Package – <Publisher/Product> – Defense` and escalate to procurement and licensing/legal for a contract-level decision.

7. Best Practices — Software License Compliance & Audit-Defense (Effective License Position)

• Pilot one publisher/product first. Prove the reconciliation on a single, well-understood product before you scale — it validates your inputs and method without fleet-wide risk.

• Evidence every entitlement. A license you cannot prove is counted as zero in an audit; one

proof-of-purchase reference per line in the `Entitlement Baseline – <Publisher/Product> – Source` is the single most valuable habit.

• Reconcile on the right metric. Per-core, per-device, and per-user products count differently; normalizing to the entitlement's metric is what makes the ELP correct.

• Apply product use rights deliberately. Downgrade, second-use, virtualization, and BYOL change the count — record the rights interpretation and its contract clause on each ELP line.

• Find your gaps before the auditor does. Build the ELP proactively and close the cheap shortfalls (reharvest, reassign, data fixes) quietly, on your terms.

• Get sign-off before spending or cutting. True-ups spend money and renewal reductions change contracts; route both through the named owners with an auditable decision.

• Reharvest surplus, do not just renew. Stale and disabled-user assignments are recoverable spend; feed surpluses to procurement as reharvest or renewal-reduction candidates.

• Reconcile on a cadence, not once. Deployment drifts; a scheduled re-reconciliation per product family keeps you audit-ready instead of audit-reactive.

• Keep the audit package fresh. Refresh the `Audit Package – <Publisher/Product> – Defense` each cycle so you can produce a defensible position on demand, not assemble one under deadline.

• Name your artifacts consistently. Reuse `Entitlement Baseline`, `Deployment Inventory`, `Effective License Position`, `Gap Register`, and `Audit Package` with the same `<Purpose> – <Scope> – <Stage>` naming so anyone can follow the trail.

• Document the method. Record the normalization mapping, metric, and rights decisions so the reconciliation is repeatable and survives an analyst handover.

8. Deliverables Summary — Software License Compliance & Audit-Defense (Effective License Position)

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	What an ELP is, the reconciliation model, common gaps, audit-defense posture
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps to build the ELP, close gaps, and assemble the audit package

Document	Purpose
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts produced

Artifact	What it holds
`Entitlement Baseline – <Publisher/Product> – Source`	What you own, with a proof reference per line
`Deployment Inventory – <Publisher/Product> – Discovered`	What you run, with source and date
`Effective License Position – <Publisher/Product> – ELP`	The reconciliation workbook (entitled vs consumed, with rights)
`Gap Register – <Publisher/Product> – Reconciled`	Shortfalls and surpluses, each with owner and action
`Audit Package – <Publisher/Product> – Defense`	The bundled, signed-off, defensible evidence set

PowerShell scripts

Script	What it does
`LICCOMP-Discovery.ps1`	Collects an installed-software / license-assignment inventory into a CSV (read-only)
`LICCOMP-Validation.ps1`	Confirms the entitlement and deployment inputs are present, normalized, and reconcilable (read-only)
`LICCOMP-Report.ps1`	Produces the ELP summary and the gap list from the reconciled inputs (read-only)
`LICCOMP-Remediation.ps1`	Safely reclaims clear over-deployment (stale/disabled-user assignments); defaults to `-WhatIf`
`LICCOMP-Set-ScopeTargeting.ps1`	Builds the reconciliation-scope group for an in-scope product (Microsoft Graph)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand.

Outcome

A defensible Effective License Position — entitlements reconciled to deployment, gaps closed, and an audit package ready — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

LICCOMP-Discovery.ps1

Purpose: Builds the "Deployment Inventory - - Discovered" input for the ELP. Reads installed-software data from a generic inventory CSV export (your discovery tool, Intune, or Configuration Manager) and/or the local machine, filters to the in-scope product, and writes a normalized per-install CSV. Read-only — it changes nothing. HOW TO RUN From a CSV export: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Discovery.ps1 ` -InventoryCsv .\inventory-export.csv -ProductName "Acme Designer" -CsvPath .\deployment.csv From the local machine (no CSV input): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Discovery.ps1 -ProductName "Acme Designer" -CsvPath .\deployment.csv EXPECTED RESULTS A console summary of matched installs by edition/version and a per-install CSV (Device, Product, Edition, Version, Source) for the Deployment Inventory artifact. .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in / CSV out. #> [CmdletBinding()] param([string]$InventoryCsv, [Parameter(Mandatory)] [string]$ProductName, [string]$CsvPath)

How to run: From a CSV export: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Discovery.ps1 ` -InventoryCsv .\inventory-export.csv -ProductName "Acme Designer" -CsvPath .\deployment.csv From the local machine (no CSV input): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Discovery.ps1 -ProductName "Acme Designer" -CsvPath .\deployment.csv EXPECTED RESULTS A console summary of matched installs by edition/version and a per-install CSV (Device, Product, Edition, Version, Source) for the Deployment Inventory artifact. .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in / CSV out. #> [CmdletBinding()] param([string]$InventoryCsv, [Parameter(Mandatory)] [string]$ProductName, [string]$CsvPath)

LICCOMP-Remediation.ps1

Purpose: Closes shortfalls and recovers surplus by removing license assignments from accounts that should not consume a license (disabled accounts, or accounts on a supplied stale-assignment list) for the in-scope product's SKU in Microsoft 365 / Entra. This is the only script that changes state, so it DEFAULTS TO -WhatIf (preview only). Nothing is removed until you re-run with -WhatIf:$false after owner sign-off (Requirement 2.6). HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Remediation.ps1 -SkuPartNumber "" -InputCsv .\stale.csv Act (after sign-off): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Remediation.ps1 -SkuPartNumber "" -InputCsv .\stale.csv -WhatIf:$false InputCsv columns: UserPrincipalName (one account per row to reclaim from) Requires Microsoft Graph SDK and scopes User.ReadWrite.All, Organization.Read.All. EXPECTED RESULTS In preview: a list of accounts and the SKU the assignment WOULD be removed from, with no change. After sign-off: those assignments removed and a per-account result. .NOTES Workplace Marketplace — Asset Mgt. State-changing; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$SkuPartNumber, [Parameter(Mandatory)] [string]$InputCsv)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Remediation.ps1 -SkuPartNumber "<SKU>" -InputCsv .\stale.csv Act (after sign-off): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Remediation.ps1 -SkuPartNumber "<SKU>" -InputCsv .\stale.csv -WhatIf:$false InputCsv columns: UserPrincipalName (one account per row to reclaim from) Requires Microsoft Graph SDK and scopes User.ReadWrite.All, Organization.Read.All. EXPECTED

RESULTS In preview: a list of accounts and the SKU the assignment WOULD be removed from, with no change. After sign-off: those assignments removed and a per-account result. .NOTES Workplace Marketplace — Asset Mgt. State-changing; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$SkuPartNumber, [Parameter(Mandatory)] [string]$InputCsv)

LICCOMP-Report.ps1

Purpose: Reconciles the entitlement baseline against the deployment inventory and reports, per product line, the Effective License Position (entitled minus consumed) and the resulting gaps. Writes the gap list for the "Gap Register - - Reconciled". Read-only — it derives a report from your inputs and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LICCOMP-Report.ps1 ` -EntitlementCsv .\entitlements.csv -DeploymentCsv .\deployment.csv -CsvPath .\gap-register.csv Entitlement CSV columns: Product, Edition, Metric, Quantity, ProofRef Deployment CSV columns: Product (one row per install/assignment/counted unit) EXPECTED RESULTS A per-product table (Entitled, Consumed, Position) and a gap CSV listing each shortfall (negative position) and surplus (positive position) for the gap register. .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in / CSV out. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$EntitlementCsv, [Parameter(Mandatory)] [string]$DeploymentCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Report.ps1 ` -EntitlementCsv .\entitlements.csv -DeploymentCsv .\deployment.csv -CsvPath .\gap-register.csv Entitlement CSV columns: Product, Edition, Metric, Quantity, ProofRef Deployment CSV columns: Product (one row per install/assignment/counted unit) EXPECTED RESULTS A per-product table (Entitled, Consumed, Position) and a gap CSV listing each shortfall (negative position) and surplus (positive position) for the gap register. .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in / CSV out. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$EntitlementCsv, [Parameter(Mandatory)] [string]$DeploymentCsv, [string]$CsvPath)

LICCOMP-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) a Microsoft Entra security group that holds the population to reconcile for one publisher/product — the members who could consume the in-scope license. Use it to define the pilot scope and to keep reconciliation consistent across cycles. Optionally seeds membership from a CSV of UPNs. Supports -WhatIf to preview without creating anything. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" -WhatIf Create the group: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" Create and seed members from a CSV (column: UserPrincipalName): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" -MembersCsv .\scope.csv Requires Microsoft Graph SDK and scopes Group.ReadWrite.All, User.Read.All. EXPECTED RESULTS A security group named "LICCOMP-Scope-" reported (created or reused) and, if a CSV is supplied, the listed members added. In -WhatIf mode, the intended actions are printed only. .NOTES Workplace Marketplace — Asset Mgt. Creates an Entra group; supports -WhatIf. #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$ProductName, [string]$MembersCsv)

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" -WhatIf Create the group: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" Create and seed members from a

CSV (column: UserPrincipalName): powershell -ExecutionPolicy Bypass -File .\LICCOMP-Set-ScopeTargeting.ps1 -ProductName "Acme Designer" -MembersCsv .\scope.csv Requires Microsoft Graph SDK and scopes Group.ReadWrite.All, User.Read.All. EXPECTED RESULTS A security group named "LICCOMP-Scope-<Product>" reported (created or reused) and, if a CSV is supplied, the listed members added. In -WhatIf mode, the intended actions are printed only. .NOTES Workplace Marketplace — Asset Mgt. Creates an Entra group; supports -WhatIf. #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$ProductName, [string]$MembersCsv)

LICCOMP-Validation.ps1

Purpose: Gate-checks the two ELP inputs before you trust the numbers. Confirms the "Entitlement Baseline - - Source" and the "Deployment Inventory - - Discovered" are present, have the required columns, carry a proof reference per entitlement line, and share at least one common product key after normalization. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LICCOMP-Validation.ps1 ` -EntitlementCsv .\entitlements.csv -DeploymentCsv .\deployment.csv Entitlement CSV expected columns: Product, Edition, Metric, Quantity, ProofRef Deployment CSV expected columns: Device/User, Product, Edition, Version EXPECTED RESULTS "INPUTS RECONCILABLE" (exit 0) when both inputs are complete and share a product key; otherwise a list of the gaps to fix (exit 1). .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$EntitlementCsv, [Parameter(Mandatory)] [string]$DeploymentCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\LICCOMP-Validation.ps1 ` -EntitlementCsv .\entitlements.csv -DeploymentCsv .\deployment.csv Entitlement CSV expected columns: Product, Edition, Metric, Quantity, ProofRef Deployment CSV expected columns: Device/User, Product, Edition, Version EXPECTED RESULTS "INPUTS RECONCILABLE" (exit 0) when both inputs are complete and share a product key; otherwise a list of the gaps to fix (exit 1). .NOTES Workplace Marketplace — Asset Mgt. Read-only. Generic CSV in. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$EntitlementCsv, [Parameter(Mandatory)] [string]$DeploymentCsv)